MAJOR PROJECT · SEMESTER VII · GROUP 07 · RCOEM CSE

Technical Reference & Viva Question Bank

Memory-Augmented Temporal Reasoning for Flood Victim Detection and Dynamic Rescue-Priority Mapping from Drone Imagery

Team Prem Baba (53) · Purva Bhoyar (57) · Adhishree Shiledar (01) · Palak Zade (68)

Guide Dr. Nisarg Gandhewar, Associate Professor

Department Computer Science & Engineering, RCOEM Nagpur

Session 2026–2027

> **How to use this document.** Part 1 is the end-to-end technical process — read it once and you can narrate the whole system. Part 2 defines every technical term the project touches, each with *what it is* and *why we chose it*; the amber notes flag the follow-up an examiner is likely to ask. Part 3 is a rapid-fire question bank. Part 4 is the honest-boundary list — the answers that protect you when a panel pushes on what the system cannot do.

PART ONE

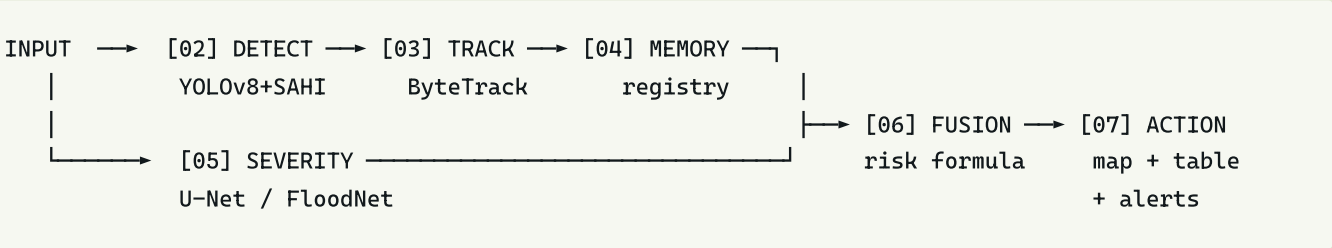
The Complete Technical Process

What actually happens, in order, from a drone switching on to a rescue boat being told where to go. Two trained deep-learning models and three logic modules.

1.1 The one-paragraph version

A drone flies over a flooded area and streams video. We decode that video into frames. Each frame goes down **two independent branches**. On the perception branch, a YOLOv8 detector — helped by SAHI slicing — finds every person, vehicle and animal, and ByteTrack gives each one a stable identity that survives across frames. That identity is the key into a registry where we keep a short rolling history per victim: how long they have been stranded, whether they have moved, and whether the water around them is rising. On the second branch, a U-Net segments the flood water and reports how severely flooded the area around each group is. The two branches meet in a weighted formula that turns five signals into a single risk score between 0 and 1, which is banded into Critical / High / Medium / Low. The output is a ranked rescue-priority map, a victim summary table and automatic alerts for critical locations.

1.2 Stage-by-stage



Stage 01 — Frame ingest

OpenCV opens the video and pulls one frame every *N* frames (a fixed **stride**). Each frame is resized so its long edge is 1280 px and stamped with a capture time. The stride is fixed on purpose: it is what converts "seen in 40 frames" into "stranded for 18 minutes". Without a known time-per-frame, the memory module has no clock.

Stage 02 — Victim detection (Model 1: YOLOv8 + SAHI)

The frame is cut into overlapping tiles — 640 px tiles with roughly 20% overlap. YOLOv8 runs on each tile independently. Because a tile is a zoomed-in crop, a victim who occupied 8 pixels in the full frame now occupies around 60 pixels, which is well inside what the detector can see. Every tile's boxes are mapped back to full-frame coordinates, and duplicates along tile seams are removed by Non-Maximum Suppression. Output: a list of boxes, each with a class (person / vehicle / animal) and a confidence.

Stage 03 — Tracking (ByteTrack)

Each existing track carries a Kalman filter that predicts where it should appear in this frame. ByteTrack matches this frame's boxes to those predictions by IoU, in two passes: high-confidence boxes first, then a second pass that rescues low-confidence boxes sitting near an unmatched prediction. A matched box updates its track. An unmatched box starts a new track with a fresh ID. An unmatched track is kept alive for up to 30 frames before being retired, so a victim briefly hidden behind a chimney does not lose their identity.

Stage 04 — Track registry and short-term memory

The registry is a dictionary keyed by `track_id`. Every frame, we append a reading — `(t, position, group count, local severity)` — into a fixed-length ring buffer holding the last K = 30 readings, then recompute three derived values: **time_stranded** = now − first_seen, **displacement** = how far the group has moved across the buffer, and **water_trend** = the slope of a straight line fitted through the buffer's severity values. This is the entire "memory" of the system, and it is deliberately short-term.

Stage 05 — Flood severity (Model 2: U-Net)

Running in parallel on the same frame, a U-Net trained on FloodNet produces a per-pixel binary mask: water or not-water. For each tracked group we take a ring of pixels around its box and compute the flooded fraction, weighted so that water closer to the group counts more. That gives one severity number per group, 0 to 1. Note the dependency: severity depends on the **frame**, not on the detections, which is why the two branches are drawn side by side rather than in series.

Stage 06 — Risk and priority fusion

Five signals are each normalised to the range 0–1 and combined in a weighted sum:

```
risk = w₁·N̂  +  w₂·S  +  w₃·T̂  +  w₄·I  +  w₅·R

      N̂ = group size (normalised)        w₁ = 0.25
      S  = flood severity                 w₂ = 0.30
      T̂ = time stranded (normalised)     w₃ = 0.20
      I  = isolation from nearest safe zone  w₄ = 0.15
      R  = rising-water trend             w₅ = 0.10

  Worked example, track 001:
      0.25(1.00) + 0.30(0.92) + 0.20(0.88) + 0.15(0.95) + 0.10(0.83)
    = 0.250 + 0.276 + 0.176 + 0.143 + 0.083  =  0.93  →  CRITICAL
```

Bands: **Critical > 0.8 · High 0.5–0.8 · Medium 0.3–0.5 · Low < 0.3.** Weights are set from domain reasoning and then validated by ablation, not learned — see Part 3, Q32–Q33.

Stage 07 — Output

Groups are sorted by descending risk and drawn as numbered, colour-coded markers on the scene. A summary table lists ID, position, headcount, risk and rank. Any group above 0.8 raises an alert. The entire output refreshes every frame, so a group whose water is rising climbs the list on its own.

1.3 What is trained and what is not

Module	Type	Trained?	Why that choice
1 · Victim detection	Deep model	Yes	Recognising a person from above is a perception problem — no rule can express it.
2 · Tracking	Algorithm	No	ByteTrack is geometry + motion prediction; it works on any detector's output.
3 · Memory	Data structure	No	Storing and differencing readings is bookkeeping, not learning.
4 · Flood severity	Deep model	Yes	Water texture varies enormously; only a learned model generalises across scenes.
5 · Risk & priority	Formula	No	Every score must be explainable when it decides who is rescued first.

PART TWO

Every Technical Term — What It Is, Why We Use It

If it appears anywhere in the report, slides or code, it is defined here. Amber notes mark the follow-up question that usually comes next.

2.1 Platform & framing

UAV `Unmanned Aerial Vehicle`

An aircraft with no pilot on board, flown remotely or autonomously — in plain speech, a drone.

Why: it is the only sensor platform that can survey a flooded region in minutes when roads are submerged and currents make ground search lethal.

Aerial / overhead imagery

Images captured looking down from height, typically 60–120 m for survey drones.

Why it matters technically: objects are tiny, top-down (so a person is an oval, not a silhouette), and the scale is uniform across the frame. Detectors trained on ground-level photos transfer badly.

Decision support system

A system that recommends an action to a human operator rather than acting itself.

Why we use this framing: the system ranks victims; a human commander dispatches. This keeps accountability where it belongs and is the honest description of what we built.

2.2 Computer vision fundamentals

Classification vs Detection vs Segmentation

Classification answers "what is in this image?" (one label per image). **Detection** answers "what is here, and where?" (boxes + labels). **Segmentation** answers "which exact pixels belong to what?" (a per-pixel label).

Why both in our project: we need detection for victims (we want to count and locate them) and segmentation for water (we need its exact extent, and water has no meaningful bounding box).

> Likely follow-up: "Why not segment the victims too?" — Because we only need position and count, and detection is far cheaper. Pixel-perfect victim outlines add nothing to a rescue ranking.

Bounding box

A rectangle around an object, stored as `(x, y, w, h)` or as two corners.

Why: it is the cheapest representation that still gives us position and size — the two things the memory and risk modules need.

Confidence score

The detector's own estimate, 0–1, that a box really contains the class it claims.

Why it matters: ByteTrack's key idea is to *not* throw away low-confidence boxes. A half-occluded victim produces a weak box, and discarding it is exactly the failure we cannot afford.

IoU `Intersection over Union`

Overlap between two boxes divided by their combined area. 1.0 = identical, 0 = no overlap.

Where we use it three separate times: (i) as the matching criterion in NMS, (ii) as the association cost in ByteTrack, (iii) as the accuracy metric for U-Net segmentation.

NMS `Non-Maximum Suppression`

A cleanup step: when several boxes overlap above an IoU threshold, keep the highest-confidence one and delete the rest.

Why we especially need it: SAHI's tiles overlap by design, so the same victim is genuinely detected two or three times. NMS at IoU 0.5 merges those duplicates back into one.

Inference vs Training

Training is fitting the model's weights on labelled data. **Inference** is running the finished model on new data.

Why the distinction matters here: SAHI is applied at *inference* time (and optionally during fine-tuning). It is not part of the network — you could remove it and the weights would be unchanged.

2.3 The detector — YOLOv8 and SAHI

YOLO `You Only Look Once`

A family of single-stage detectors that predict all boxes and classes in one forward pass over the image, instead of first proposing regions and then classifying them.

Why single-stage: speed. Two-stage detectors like Faster R-CNN are marginally more accurate but far slower — and rescue is a real-time problem.

YOLOv8

The Ultralytics release of YOLO used here. It is **anchor-free** (predicts box centres directly rather than adjusting predefined anchor shapes) and has a decoupled head that predicts classification and box regression on separate branches.

Why v8 specifically: mature tooling, one-line training API, excellent documentation, huge community, and native export formats. For a student project, tooling maturity is a real engineering criterion.

> Likely follow-up: "Why not YOLOv11 / v12?" — Perfectly valid alternatives; v8 is chosen for stability and documentation. The architecture is swappable in one line, and comparing versions is a natural extension.

Backbone · Neck · Head

The three parts of a detector. The **backbone** (CSPDarknet in v8) extracts features. The **neck** (PANet-style) fuses features across scales. The **head** outputs boxes and classes.

Why the neck matters for us: multi-scale fusion is what lets one network detect both a large vehicle and a tiny person. Small objects are resolved by the high-resolution early feature maps.

The small-object problem

Detectors lose objects below roughly 32×32 pixels, because repeated downsampling in the backbone shrinks a small object's features until nothing distinctive survives.

Why this is the core of our project: from 100 m altitude a person is around 8–15 px. This is simultaneously the hardest technical problem and the most rescue-critical one.

SAHI `Slicing Aided Hyper Inference`

A wrapper, not a network. It cuts a large image into overlapping tiles, runs the detector on each tile, maps every detection back to full-frame coordinates, and merges the results with NMS.

Why it works: the detector always sees a fixed input size. Feeding it a 640 px crop of a 1280 px frame effectively doubles the object's apparent size. A 10 px victim becomes a 20–60 px victim — comfortably detectable.

> Likely follow-up: "What does it cost?" — Inference time. Running 12 tiles instead of 1 frame is roughly an order of magnitude slower. That is the honest trade-off, and it is why we report FPS separately.

Tile size & overlap ratio

SAHI's two main knobs. We use ~640 px tiles with ~20% overlap.

Why overlap at all: without it, a victim sitting exactly on a tile boundary is cut in half and detected by neither tile. Overlap guarantees every object appears whole in at least one tile.

Occlusion

An object partly hidden — by a chimney, a tree, debris, or another person.

How we handle it: occlusion-aware fine-tuning (training on flood images that actually contain partial victims) plus ByteTrack's low-confidence pass, which keeps an ID alive through a weak detection.

2.4 Training concepts

Transfer learning

Starting from weights already trained on a different, larger dataset instead of from random values.

Why: the early layers of any vision network learn edges, corners and textures — universal features. Reusing them means we need far less flood data and far less compute.

Pre-training and Fine-tuning

Pre-training is the first, large, general training run. **Fine-tuning** is a second, shorter run on the specific target data, usually at a lower learning rate.

Our two-stage design: pre-train on VisDrone so the model learns "what people and vehicles look like from drone height"; fine-tune on flood imagery so it learns "...against water, rooftops and debris".

> Likely follow-up: "Why not just train on flood data alone?" — Too few flood images to learn overhead appearance from scratch; it would overfit. Proving this is exactly ablation #1 in our evaluation plan.

Why a *lower* learning rate when fine-tuning

The model already sits in a good region of weight space. A large learning rate would destroy the general knowledge learned in stage 1 — an effect called catastrophic forgetting.

Epoch · Batch size · Iteration

An **epoch** is one full pass over the training set. **Batch size** is how many images are processed before the weights update. An **iteration** is one such update.

Practical note: batch size is limited by GPU memory. At 1280 px input we may only fit batches of 4–8, which is why gradient accumulation is sometimes used.

Optimizer — SGD vs AdamW

The algorithm that decides how to change weights given the gradients. **SGD** with momentum is simple and generalises well; **AdamW** adapts the step size per parameter and converges faster.

Our use: Ultralytics' default (AdamW for short runs, SGD for long ones) is a sound choice; we tune it empirically rather than dogmatically.

Learning-rate warmup and decay

Warmup ramps the learning rate up over the first few epochs; **decay** lowers it towards the end (typically cosine).

Why: warmup prevents huge early updates from wrecking pre-trained weights; decay lets the model settle precisely into a minimum instead of bouncing around it.

YOLOv8 loss — three components

Box loss (CIoU) penalises geometric mismatch between predicted and true boxes. **Classification loss** (BCE) penalises wrong labels. **DFL** (Distribution Focal Loss) refines box edges by predicting a distribution over positions rather than a single value.

Why three: a detector must get location *and* identity right; each term supervises a different failure.

Data augmentation — mosaic, flip, HSV, scaling

Random transformations applied to training images. **Mosaic** stitches four images into one; **HSV jitter** shifts hue, saturation and brightness; plus horizontal flips and random scaling.

Why mosaic is unusually valuable here: it packs four scenes into one frame, which shrinks every object — so the model sees far more small-object examples than the dataset actually contains.

> Likely follow-up: "Which augmentation would you avoid?" — Vertical flips and heavy rotation. Overhead scenes have a real gravity direction (roof ridges, shadows, water flow); destroying it hurts.

Overfitting and Underfitting

Overfitting: the model memorises the training set and fails on new data (train loss falls, validation loss rises). **Underfitting**: it fails on both — too little capacity or training.

How we detect it: watch the gap between training and validation mAP. Our defences are augmentation, transfer learning and early stopping.

Train / Validation / Test split

Training data fits weights; validation data tunes hyperparameters and triggers early stopping; the test set is touched once, at the end.

Why strictly separated: if you choose hyperparameters using the test set, your reported number is optimistic and the result is not honest.

Class imbalance

When one class hugely outnumbers others — in VisDrone, vehicles vastly outnumber animals.

How we address it: merge VisDrone's many vehicle sub-classes into a single "vehicle" class, and report per-class AP so a strong vehicle score cannot hide a weak person score.

YOLO annotation format

One `.txt` per image; one line per object: `class_id x_centre y_centre width height`, all normalised to 0–1.

Why conversion is a real work item: VisDrone ships a different format (absolute corner coordinates plus occlusion and truncation flags), so a converter script is a genuine deliverable, not a footnote.

2.5 Tracking

MOT `Multi-Object Tracking`

Following many objects at once across frames while keeping each one's identity consistent.

Tracking-by-detection

The dominant MOT paradigm: run a detector independently on every frame, then link detections across frames. The tracker never looks at the image itself.

Why this suits us: it cleanly decouples the two problems. We can improve the detector without touching the tracker, and vice versa.

Track ID

An integer assigned to an object the first time it is seen and preserved for as long as it is followed.

Why it is the hinge of the whole project: it is the key into the memory registry. Without a stable ID there is nothing to attach history to — and with no history there is no temporal reasoning, which is the project's headline claim.

Kalman filter

A recursive estimator that predicts an object's next state (position, velocity, box size) and then corrects that prediction using the new measurement, weighting each by its uncertainty.

Why: matching by prediction is far more reliable than matching by nearest box. It also carries a track through a frame where the detector fails entirely.

Data association & the Hungarian algorithm

The problem of deciding which detection belongs to which existing track. Built as a cost matrix (usually 1 − IoU) and solved optimally by the Hungarian / Jonker-Volgenant algorithm.

ByteTrack — and its one big idea

Most trackers discard detections below a confidence threshold. ByteTrack keeps them and matches in **two passes**: high-confidence boxes are associated first, then the leftover low-confidence boxes are matched against still-unmatched tracks.

Why this is the right tracker for flood rescue: occlusion and tiny size are exactly what produce low-confidence boxes. A conventional tracker drops the ID and restarts the "time stranded" clock at zero; ByteTrack keeps it running.

> Likely follow-up: "Why not DeepSORT?" — DeepSORT adds an appearance-embedding network, which costs extra compute and works poorly on 10-pixel targets that have almost no appearance to embed. ByteTrack needs no extra network and is faster.

ID switch

A tracking failure where two objects swap identities, or one object is given a new ID after an occlusion.

Why we measure it: an ID switch silently resets that victim's memory. IDF1 is the metric that exposes this.

Track birth, death and max_age

A new ID is created for an unmatched detection; a track that goes unmatched for `max_age` frames (we use ~30) is retired.

Why not retire immediately: a victim hidden behind a chimney for one second must keep their identity, or their accumulated history is thrown away.

2.6 Memory and temporal reasoning

Temporal reasoning

Drawing conclusions from how a situation *changes* over time, not only from its current state.

The one-sentence justification: "8 people on a roof" and "8 people on a roof with water rising steadily for nine minutes" are identical to a single-frame system and completely different rescue problems.

Ring buffer `circular buffer`

A fixed-length array that overwrites its oldest entry when full. We hold K = 30 readings per victim.

Why fixed-length: constant memory per victim regardless of how long the operation runs, and it naturally implements "short-term" — old readings expire on their own.

The track registry

A Python dictionary mapping `track_id → {first_seen, last_seen, buffer[K], derived fields}`, persisted to JSON or SQLite once per session.

Why not a real database: there is one process, one operator, and at most a few hundred tracks. A server database would add deployment complexity and buy nothing.

> Say "in-process store persisted to JSON/SQLite", **not** "database". The word "database" invites questions about schema, indexing and concurrency that your design does not need to answer.

Derived fields — time_stranded, displacement, water_trend

time_stranded = current time − first_seen. **displacement** = distance between the oldest and newest position in the buffer. **water_trend** = slope of a least-squares line fitted through the buffer's severity values.

Why displacement matters: a group that has not moved in fifteen minutes is trapped; a group that is moving may be self-rescuing and needs a lower priority.

Why "rising water" needs a steady viewpoint

The trend compares severity readings across frames. If the drone moves between readings, "more water" may simply mean "the camera panned towards the river".

Our assumption, stated openly: a hovering or slowly-orbiting drone, or frame alignment (homography registration) as a mitigation. This is a genuine limitation and we declare it rather than hide it.

2.7 Segmentation — U-Net

Semantic segmentation

Assigning a class label to every pixel. Ours is binary: water or not-water.

U-Net

An encoder–decoder network shaped like a U. The **encoder** downsamples, capturing "what is here"; the **decoder** upsamples back to full resolution, recovering "exactly where". **Skip connections** copy each encoder stage directly across to the matching decoder stage.

Why skip connections are the whole point: downsampling destroys precise boundaries. The skips re-inject that lost spatial detail, which is what gives U-Net sharp edges.

> Likely follow-up: "Why U-Net and not DeepLab or SegFormer?" — U-Net trains well on small datasets (it was designed for biomedical data, where labels are scarce), is simple to implement, and FloodNet is not large. Heavier architectures need more data to beat it.

Dice loss and cross-entropy

Cross-entropy scores each pixel independently. **Dice loss** scores the overlap of the whole predicted region against the truth. We combine them.

Why combine: cross-entropy alone is dominated by the majority class — if 80% of pixels are water, predicting "all water" scores well. Dice is insensitive to that imbalance and forces correct shape.

Mask

The binary output image: 1 where the pixel is water, 0 otherwise.

From mask to a severity number

For each group, take an annulus around its bounding box, compute the fraction of those pixels that are water, and weight nearer pixels more heavily. Result: one number in 0–1.

Why an annulus rather than the box itself: the box contains the rooftop the victims are standing on, which is dry by definition. The danger is the water *around* them.

2.8 Fusion and scoring

Normalisation (min–max)

Rescaling a raw quantity into 0–1 via `(x - min) / (max - min)`.

Why essential: group size (0–50 people) and severity (0–1) live on wildly different scales. Summing them unnormalised would let headcount silently dominate every other signal.

Weighted sum / linear scoring model

Multiply each normalised signal by a weight and add. Weights sum to 1, so the result stays in 0–1.

Why not a neural network: there is no labelled ground truth for "correct rescue priority" — no dataset records which group a real commander saved first. With no labels, there is nothing to train on. A transparent formula is both the honest and the defensible choice.

Priority bands

Critical > 0.8 · High 0.5–0.8 · Medium 0.3–0.5 · Low < 0.3.

Why band a continuous score at all: humans act on categories under pressure. "0.81 versus 0.78" is not an instruction; "Critical versus High" is.

Ablation study

Removing one component and re-measuring, to isolate how much that component actually contributed.

Our four ablations: ±SAHI, ±flood fine-tuning, ±memory, ±severity. This is how we prove each module earns its place rather than merely asserting it.

2.9 Evaluation metrics

Metric	What it measures	Why we report it
Precision	Of everything flagged, how much was real — TP / (TP+FP)	Low precision floods the commander with false alarms.
Recall	Of everything real, how much was found — TP / (TP+FN)	**The metric that matters most in rescue.** A missed victim is a life.
AP / mAP	Area under the precision–recall curve, averaged over classes	The single standard number for comparing detectors.
mAP@0.5	mAP counting a box correct at IoU ≥ 0.5	The lenient, widely-quoted figure — comparable with published work.
mAP@0.5:0.95	Averaged over IoU thresholds 0.50 to 0.95	Stricter; rewards precise localisation, not just rough hits.
AP_{small}	AP restricted to objects under 32×32 px	**Our headline metric.** It is the number SAHI is supposed to move.
IoU / mIoU	Pixel overlap of predicted vs true mask	The standard segmentation score, for U-Net.
MOTA	Combined tracking accuracy: misses + false positives + ID switches	Overall tracking health in one number.
IDF1	How consistently the correct identity is kept over a track's life	**More important than MOTA for us** — identity consistency is what memory depends on.
FPS	Frames processed per second	Needed to justify any "near real-time" claim, especially with SAHI's overhead.

The precision–recall trade-off, in rescue terms. Lowering the confidence threshold raises recall (fewer victims missed) and lowers precision (more false alarms). For most applications that is a bad trade. For search and rescue it is the right one: a false alarm costs a boat a few minutes; a missed detection costs a life. Say this if asked which you would tune for — it shows you understand the domain, not just the metric.

2.10 Datasets

Dataset	What it is	Role in our project
VisDrone	Large aerial detection & tracking benchmark from drones — urban scenes, people and vehicles, ~10 k images	**Stage 1 pre-training** for Model 1. Teaches general overhead appearance.
Roboflow flood set	Flood-specific detection data — people on rooftops, in boats and in water, plus vehicles and animals; ships in YOLO format	**Stage 2 fine-tuning** for Model 1. Teaches flood context.
FloodNet	High-resolution post-flood UAV imagery with segmentation masks	**Training data** for Model 2 (U-Net water segmentation).
C2A	Synthetic UAV disaster human-detection dataset	Cited as related work; a fallback source of extra training images if flood data proves thin.

> **Know this weakness before the panel finds it.** There is no public benchmark of *flood video with labelled stranded people*. Model 1 is therefore fine-tuned on flood **images**, while tracking and memory are demonstrated on **video**. Say this plainly, and note that releasing such a benchmark is listed in our future scope — that turns a weakness into a contribution.

2.11 Tools and environment

Tool	What it does for us
PyTorch	The deep-learning framework — autograd, GPU tensors, training loops
Ultralytics	YOLOv8 implementation, training API, metric reporting, export
SAHI	Slicing wrapper around inference and fine-tuning
OpenCV	Video decode, resize, drawing overlays, writing output video
NumPy	Array maths — buffers, normalisation, trend fitting
Albumentations	Augmentation pipeline for the U-Net (YOLO has its own built in)
CUDA / cuDNN	GPU acceleration — without it, training is impractical
Google Colab	Free/cheap GPU access for training runs
Matplotlib / Folium	Rendering the priority map and summary visuals
Git / GitHub	Version control and the documented-repository deliverable

PART THREE

Viva Question Bank

Forty questions a panel is likely to ask, with answers you can give in one or two sentences.

Scope and novelty

Q1 In one sentence, what does your project do?

It detects stranded flood victims from drone footage, tracks them over time, remembers how each one's situation is changing, measures how flooded their surroundings are, and produces a ranked rescue-priority map telling teams who to save first.

Q2 What is novel here? You are only combining existing tools.

The novelty is the integration and the output, not a new network. No existing flood system chains detection, tracking, temporal memory, flood-severity fusion and rescue prioritisation together. Prior work stops at boxes, masks or counts; ours ends at a decision. Integration that produces a decision-useful output is a recognised, legitimate form of contribution.

Q3 Why is this better than existing UAV flood systems?

Existing systems describe the present frame. Ours reasons across time and returns an ordered action list. Practically, that means it can tell a commander that a smaller group whose water is rising fast outranks a larger group on safe high ground — a judgement no single-frame system can make.

Q4 How many models have you actually trained?

Two. YOLOv8 for victim detection and U-Net for flood-water segmentation. Tracking, memory and risk scoring are algorithmic — no training involved — and that is a deliberate design decision, not a shortcut.

Detection

Q5 Why YOLO and not Faster R-CNN?

YOLO is single-stage and much faster. Rescue is time-critical, and we already pay a large speed penalty for SAHI slicing — a two-stage detector on top of that would be unusable.

Q6 Why YOLOv8 rather than a newer version?

Stability and tooling. v8 has mature documentation, a stable API and a large community, which matters for a project we must debug ourselves. The architecture is swappable in one line, so comparing newer versions is a natural extension.

Q7 What exactly does SAHI do?

It slices the frame into overlapping tiles, runs the detector on each tile, maps the boxes back to full-frame coordinates and merges them with NMS. Because a tile is a zoomed crop, a tiny victim occupies many more pixels when the detector sees them.

Q8 Is SAHI a model? Do you train it?

No. It is an inference wrapper around the detector. The network weights are unchanged by it. It can optionally also be used to generate sliced training data during fine-tuning.

Q9 What does SAHI cost you?

> Inference time — roughly an order of magnitude, since we run twelve tiles instead of one frame. That is why FPS is reported as a separate metric and why any real-time claim is made carefully.

Q10 Why do the tiles overlap?

> So that a victim lying exactly on a tile boundary is not cut in half and missed by both tiles. Overlap guarantees every object appears whole somewhere; NMS then removes the resulting duplicates.

Q11 Why is small-object detection hard?

> Repeated downsampling in the backbone shrinks a small object's features until nothing distinctive remains. Below roughly 32×32 pixels, detectors degrade sharply — and from 100 m a person is 8–15 pixels.

Q12 Which three classes and why?

> Person, vehicle and animal. People are the rescue priority; vehicles indicate stranded occupants and block routes; animals matter for livelihood in rural floods and are explicitly part of Indian disaster response.

Training

Q13 Why two-stage training?

> VisDrone teaches general overhead appearance from a large dataset; flood data teaches the specific appearance of victims against water and rooftops from a small one. General-then-specific generalises better than either dataset alone, and we prove it with an ablation.

Q14 Why not train only on flood data?

> There is not enough of it. Learning overhead object appearance from scratch on a small flood set would overfit badly. Transfer learning lets us inherit that knowledge instead of relearning it.

Q15 What is transfer learning, in one line?

> Starting from weights trained on a larger related dataset instead of from random initialisation, so the model reuses general visual features and needs far less target data.

Q16 Why a lower learning rate during fine-tuning?

> To avoid catastrophic forgetting. The model is already in a good region of weight space; large updates would destroy the general knowledge learned in stage one.

Q17 Which augmentations, and which would you avoid?

> Mosaic, horizontal flip, random scaling and HSV jitter. Mosaic is especially valuable because packing four scenes into one frame shrinks every object, generating extra small-object examples. We avoid vertical flips and heavy rotation — overhead scenes have a real gravity direction in roof ridges, shadows and flow.

Q18 How will you know if you have overfitted?

> Training mAP keeps climbing while validation mAP flattens or falls. Our defences are augmentation, transfer learning and early stopping on validation performance.

Q19 What are the three terms in YOLOv8's loss?

> Box regression loss (CIoU), classification loss (BCE) and Distribution Focal Loss, which refines box edges by predicting a distribution over positions rather than a single value.

Tracking and memory

Q20 Why do you need tracking at all?

> Because without a stable identity there is nothing to attach history to. "How long has this person been stranded" is meaningless unless the system knows it is the same person as a minute ago.

Q21 Why ByteTrack rather than SORT or DeepSORT?

> ByteTrack also associates low-confidence detections in a second pass, which is exactly the case that occlusion and tiny size produce. DeepSORT needs an appearance-embedding network that costs compute and performs poorly on 10-pixel targets with almost no appearance to embed.

Q22 What does the Kalman filter do here?

> It predicts where each track should appear in the next frame and then corrects that prediction with the new measurement. This makes association far more reliable and carries a track through frames where the detector fails.

Q23 What exactly does "memory" store?

> Per track ID: first-seen time, last-seen time, and a ring buffer of the last 30 readings of position, group count and local severity — from which time stranded, displacement and water trend are derived.

Q24 Is your memory a neural network — an LSTM?

> No, it is an explicit data structure. An LSTM would need labelled sequences of "correct" temporal behaviour to train on, which do not exist for this problem. A buffer with derived statistics is transparent, needs no data, and every number in it can be explained.

Q25 Where is the data stored?

> In an in-process registry — a Python dictionary — persisted to JSON or SQLite once per session. With one operator and a few hundred tracks, a server database would add deployment complexity for no benefit.

Q26 How do you know the water is rising?

> We fit a straight line through the severity readings in the buffer and read its slope. A positive slope means rising water. This assumes a steady drone viewpoint — a limitation we state explicitly, with frame registration as the mitigation.

Q27 What happens if the tracker loses an ID?

> That victim's history restarts, so their time-stranded and trend terms reset and their priority drops temporarily. That is exactly why we measure IDF1 and why ByteTrack's low-confidence pass was chosen.

Segmentation and fusion

Q28 Why a second model instead of adding a water class to YOLO?

> Water has no meaningful bounding box — it is an irregular region covering most of the frame. We need its exact pixel extent to measure how flooded the area around each group is, and that is segmentation, not detection.

Q29 Why U-Net?

> Its skip connections restore the spatial detail that downsampling destroys, giving sharp boundaries, and it was designed for small datasets — which matches FloodNet's size. Heavier architectures need more data to beat it.

Q30 Why Dice loss and not just cross-entropy?

> Cross-entropy is dominated by the majority class — if 80% of pixels are water, "predict all water" scores well. Dice measures region overlap and is insensitive to that imbalance, so it forces correct shape.

Q31 How do you turn a mask into a severity number?

> Take a ring of pixels around the group's box — not the box itself, which contains the dry rooftop — and compute the distance-weighted fraction that is water. That gives one number between 0 and 1.

Q32 How did you choose the risk weights?

> From domain reasoning — severity and group size dominate, trend is a modifier — then validated by ablation and sensitivity checks. They are not learned, because no dataset labels "correct" rescue priority.

Q33 Why not learn the weights with a neural network?

> There is no ground truth to learn from; no dataset records which group a real commander saved first. And a transparent formula can be justified line by line, which matters when the output decides who is rescued.

Q34 Why band the score instead of using the raw number?

> Because people act on categories under pressure. "0.81 versus 0.78" is not an instruction; "Critical versus High" is.

Evaluation

Q35 Which metric matters most, and why?

> Recall, and specifically AP on small objects. In rescue, a false alarm costs a boat a few minutes; a missed detection costs a life. We tune the confidence threshold accordingly and say so.

Q36 What is mAP@0.5 versus mAP@0.5:0.95?

> The first counts a box correct at IoU ≥ 0.5. The second averages over thresholds from 0.50 to 0.95 in steps of 0.05, so it also rewards precise localisation rather than rough hits.

Q37 How will you prove SAHI actually helped?

> By running the identical model with and without slicing on the identical test set and comparing AP_{small}. That is a controlled ablation — only one variable changes.

Q38 How will you prove memory helped, when there is no ground truth for priority?

> Two ways. Quantitatively, we compare ranking stability and ordering against a hand-labelled set of scenarios where the correct order is unambiguous. Qualitatively, we show concrete cases where the memory-enabled ranking differs from the single-frame ranking and explain why the memory-enabled one is right.

Q39 Can you compare against RescueNet's 98% mAP?

> Not directly — that number comes from a different dataset, and comparing across datasets is not a fair comparison. We re-run a baseline detector on *our* data for a controlled comparison, and cite published figures only as context.

Q40 Does it run in real time?

> Not at full video rate with SAHI enabled — slicing costs roughly an order of magnitude. But rescue does not need 30 fps: a scene update every one to two seconds is operationally sufficient, and we report measured FPS rather than claiming real-time.

PART FOUR

Honest Boundaries — What to Say When Pushed

Every one of these is a limitation an examiner can find. State it first and it becomes evidence of rigour; get caught by it and it becomes a weakness.

If they ask…	Say this
"Do you get GPS coordinates?"	No. We produce **relative pixel positions** within the frame — bounding-box centres — which is enough to rank and map victims against each other. True latitude/longitude needs the drone's GPS and camera pose, which the public datasets do not ship. It is listed as future work, and it is a data problem, not an algorithmic one.
"Can it predict future flood levels?"	No, and we deliberately do not claim it. We report **current severity plus an observed rising trend**. Forecasting water levels needs hydrological data no image dataset contains.
"Can it tell which floor the water reached?"	No. A top-down view shows water **extent** around a building, not vertical height against it. The floor analogy is useful to explain the idea, but it is not what the model measures.
"Why not thermal cameras? They would find people at night."	They would, and it is the single most valuable extension. We use RGB only because **no paired flood dataset with thermal imagery exists** to train on. Listed as future work.
"What about people under tree canopy?"	Out of scope, and honestly so. Optical cameras cannot see through canopy — that is a physics limit, not a model limit. Our scope statement says open and semi-open flood scenes.
"Your memory only holds 30 frames. Is that really 'memory'?"	It is **short-term memory**, and we call it that everywhere. Thirty readings at our stride covers the window over which flood conditions meaningfully change. Long-term historical-flood memory is separate future work.
"Where is your flood-victim benchmark?"	There is not one — that is a documented gap in the field, and every flood-rescue paper we surveyed uses its own data. We fine-tune on flood **images** and demonstrate tracking and memory on **video**, and we state that split openly. Releasing such a benchmark is in our future scope.
"How do you know the water is rising and not that the drone moved?"	We assume a steady or slowly-orbiting viewpoint, and frame registration is the mitigation. This is a real assumption and we declare it rather than hide it.
"Are these your results?"	Not yet. The priority-map and detection visuals in the presentation are **illustrative mockups of the intended output**. Quantitative results follow training and testing in Phases 3–5.

> **The single most useful habit for the viva.** When you hit a limitation, name it, explain *why* it exists (usually: no dataset supports it), and say where it sits in the future-scope list. A panel is not testing whether your system is perfect — it is testing whether you know where its edges are.

Thirty-second summary, for the opening

> "During a flood, rescue teams face two problems: they cannot reach large areas, and they do not know how many people are stranded or where. Drones solve the first. Our project solves the second — and goes further.
>
> We built a pipeline of two trained models and three logic modules. YOLOv8 with SAHI slicing detects victims, including the tiny occluded ones standard detectors miss. ByteTrack gives each victim a stable identity across frames. A memory module remembers how long each has been stranded and whether the water around them is rising. A U-Net measures flood severity. All of it fuses into one risk score per location, ranked Critical to Low.
>
> The result is not a detection — it is a rescue order. That is what no existing flood system produces, and it is our contribution."